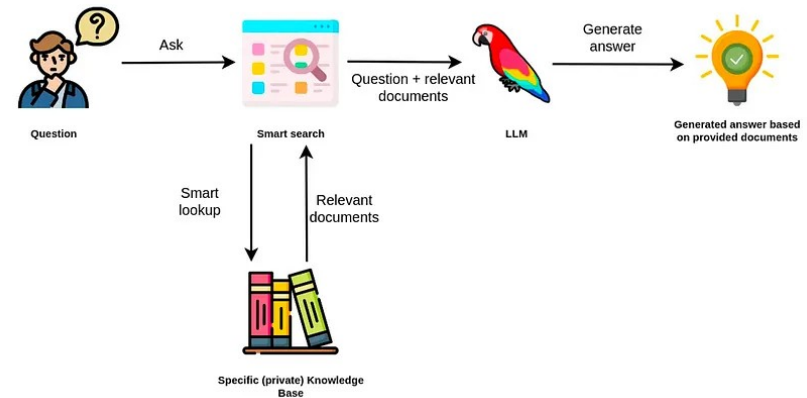


The background is a dark blue gradient with intricate white and light blue circuit-like patterns. A central silhouette of a human head is visible, with the brain area filled with a complex network of glowing nodes and lines, suggesting neural networks or AI processing. The overall aesthetic is high-tech and futuristic.

Retrieval-Augmented Generation (RAG) Chains mit LangChain

Retrieval-Augmented Generation (RAG)

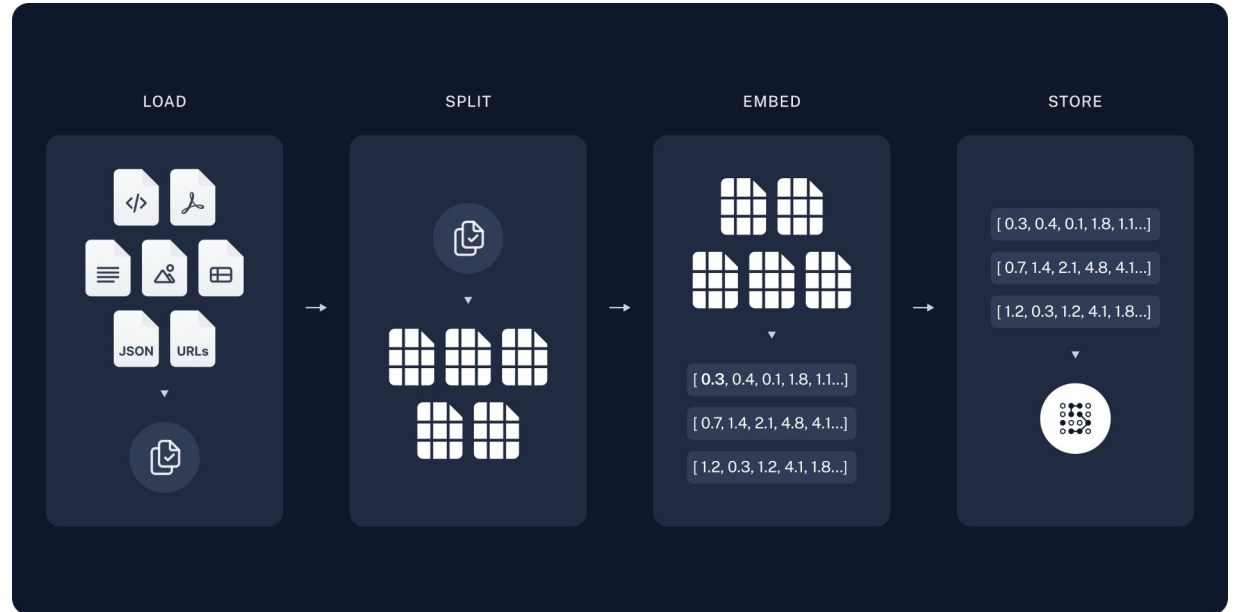
- Kombiniert LLMs mit externen Wissensquellen.
- Klassische LLMs beziehen ihr Wissen aus den Trainingsdaten - oft veraltet oder unvollständig.
- RAG-System kann gezielt auf aktuelle, domänenspezifische und verlässliche Informationen zugreifen.
- Retrieval-Komponente (zum Auffinden relevanter Informationen) + Generierungs-Komponente (zum Formulieren der Antwort).
- Zugriff auf aktuelle und domänenspezifische Daten
- Reduzierte Halluzinationen (weniger erfundene Fakten)
- Effiziente Wissensintegration (kein teures Nachtrainieren nötig)



Klassische RAG-Architektur

Indexierungspipeline:

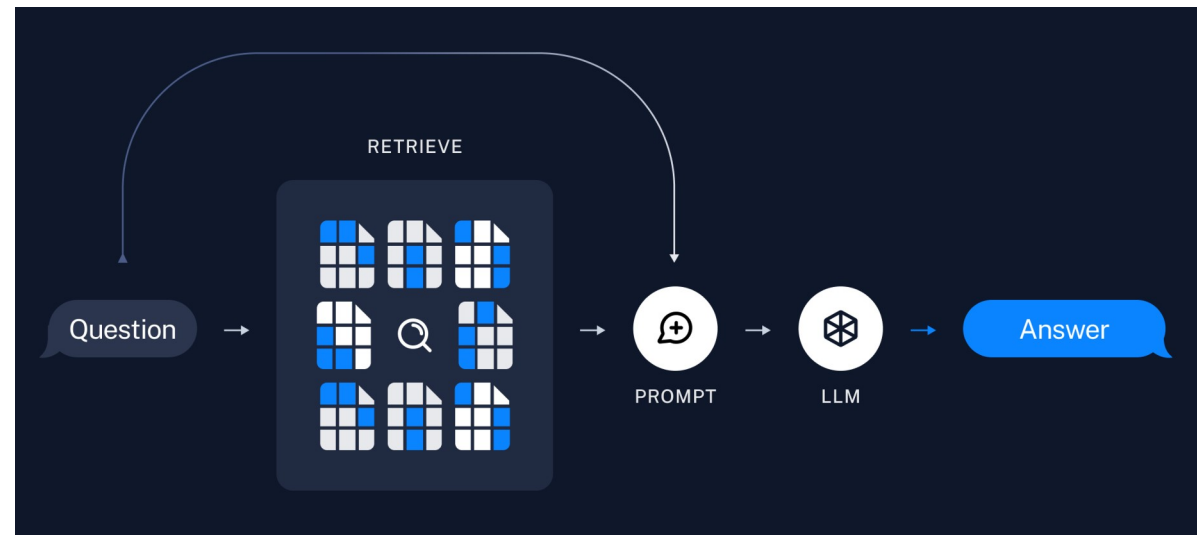
- **Laden:** Dokumente werden mit Loaders eingelesen.
- **Teilen:** Die Dokumente werden in handhabbare Chunks aufgeteilt.
- **Einbetten & Speichern:** Die Chunks werden in Vektor-Embeddings umgewandelt und in einer Vektordatenbank gespeichert (z. B. FAISS, Chroma).



Klassische RAG-Architektur

Retrieval- und Generierungskette:

- **Abrufen:** Für eine Nutzeranfrage werden relevante Dokument-Chunks mit einem Retriever gefunden.
- **Generieren:** Das Sprachmodell erhält die abgerufenen Kontexte sowie die Frage und erzeugt darauf basierend eine fundierte Antwort.



Wichtige Parameter

Retriever

- **top_k / Anzahl der zurückgegebenen Chunks**
- **Chunk-Größe (Chunk-Length)** - Länge der Textabschnitte; zu klein → Kontext-Verlust, zu groß → Rauschen.
- **Chunk-Overlap (Überlappung)** - Wie stark sich benachbarte Chunks überlappen, damit wichtige Sätze nicht „zerhackt“ werden.
- **Suchalgorithmus (BM25 vs. Vektorsuche)** - Entscheidung zwischen sparse-Search (Stichwort-basiert, z.B. BM25) und dense-Search (Vektoren/Embeddings).
- **Embedding-Modell** - Welches Modell die Chunks und die Anfrage in Vektoren umwandelt.

Weitere Parameter im RAG-Flow

- Hybrid-Suche ein/aus - Kombination von Stichwort- und Vektorsuche, um Präzision und Recall zu verbessern.
- Reranking-Strategie - Ob und wie nachträglich die Treffer nach Relevanz neu sortiert werden (z.B. mit einem kleineren Modell).
- Sprache, Herkunft, Datum oder anderen Metadaten, um nur relevante Dokumente zu berücksichtigen.

Prompt-Design

- Wenn das Prompt nicht klar genug ist, kann das LLM anfangen zu „halluzinieren“ – also Informationen zu erfinden, die nicht im Kontext stehen.
- Prompts explizit formulieren, dass das LLM angewiesen wird, ausschließlich die bereitgestellten Informationen (retrieved context) zu verwenden.

„Beantworte die folgende Frage nur auf Basis des bereitgestellten Kontexts. Wenn die Information fehlt, sage: ‚Im Kontext liegt dazu keine Information vor.‘“